

第3章 输入输出与字符串



● 学习目标



- ◆ 掌握Python的基本输入和输出
- ◆ 掌握常用的字符串方法

- ◆ Python的基本输入
- ◆ Python的基本输出
- ◆ 字符串

(1) 标准输入

`input ( " 提示字符串" )`

程序执行到`input`函数时先在屏幕上显示提示字符串，然后停下来等待用户的键盘输入，函数接收从键盘输入的任意一个字符串，将所有输入默认为字符串处理。

```
In [90]: ▶ x=input("Please enter your input:")  
          type(x)
```

```
Please enter your input:5
```

```
Out[90]: str
```

● 数据的输入与输出

- `input()` 函数的返回结果都是字符串，实际使用时需要将其转换为相应的类型再进行处理。

```
In [91]: ► x=int(input("Please enter your input:"))  
          type(x)
```

```
Please enter your input:5
```

```
Out[91]: int
```

(2) 标准输出

print()函数的格式如下：

```
print([objects][, sep=' '][, end='\n'][,file=sys.stdout][,flush=False])
```

objects是输出的对象，多个对象之间用英文逗号分割；

sep是对象之间插入的分隔符，默认是空格；

end是添加在输出文本最后的一个字符，默认是换行符；

file指定输出内容发送到的文件，默认是显示器；

flush指定输出的内容是否立即写文件。

(2) 标准输出

```
print([objects][, sep=' '][, end='\n'][,file=sys.stdout][,flush=False])
```

参数说明:

(1) [object1,...]待输出的对象，表示可以一次输出多个对象，输出多个对象时，需要用“,”分隔，会依次打印每个object，遇到逗号“,”会输出一个空格。

```
In [168]: ► a1, a2, a3="aaa", "bbb", "ccc"
           print(a1, a2, a3)

           aaa bbb ccc
```

(2) 标准输出

```
print([objects][, sep=' '][, end='\n'][,file=sys.stdout][,flush=False])
```

参数说明：

(2) `sep=""`用来间隔多个对象，默认值是一个空格，还可以设置成其它字符。

```
a1, a2, a3="aaa", "bbb", "ccc"  
print(a1, a2, a3, sep="***")
```

```
aaa***bbb***ccc
```

(2) 标准输出

`print([objects][, sep=' '][, end='\n'][,file=sys.stdout][,flush=False])`

参数说明:

(3) `end="\n"`参数用来设定以什么结尾，默认值是换行符，也可以换成其它字符串，用这个选项可以实现不换行输出，如使用 `end=" "`。

```
In [169]: ▶ a1, a2, a3="aaa", "bbb", "ccc"
           print(a1 ,end="@")
           print(a2 ,end="@")
           print(a3)
```

aaa@bbb@ccc

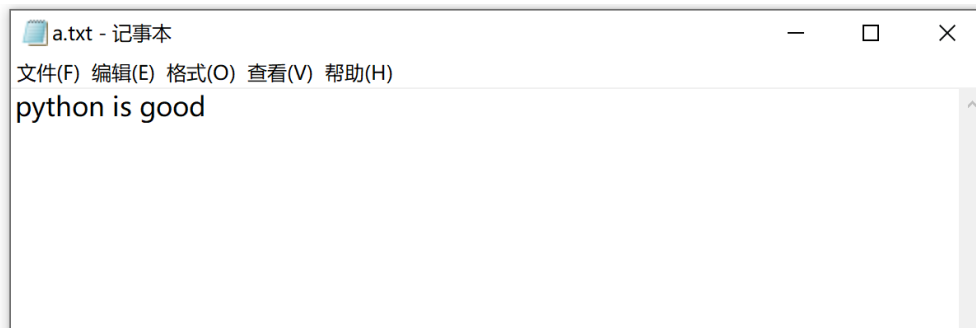
(2) 标准输出

`print([objects][, sep=' '][, end='\n'][,file=sys.stdout][,flush=False])`

参数说明：

(4) 参数file设置把print中的值打印到什么地方，可以是默认的系统输出sys.stdout，即默认输出到终端，可以设置file=文件储存对象，把内容存到该文件中。

```
In [171]: ► f = open('a.txt', 'w')           #以写的方式打开文件a.txt  
           print('python is good', file=f)  
           f.close()                        #关闭文件
```



更多例子:

```
In [94]: ▶ x, y=5, 9  
print(x, y, sep=',', end='!')
```

5, 9!

```
In [74]: ▶ out=open("test.txt", "w")  
print("Helloworld", file=out)
```



格式化输出

01

%格式化运算符

02

format()函数

03

Raw字符串

● 数据的输入与输出

- 字符串格式化%输出：字符串格式化表达式用%表示，%之前是需要进行格式化的字符串，%之后是需要填入字符串中的实际参数。语法如下：“%格式控制符”%实际参数

“%格式控制符”%实际参数

- print()函数可使用一个字符串模板进行格式化输出。
- 模板中有**格式符**，这些格式符**为真实值输出预留位置**，并**说明真实数值应该呈现的格式**。

```
In [110]: ▶ x=520
           print("sum=%d"%x)

           sum=520
```

➤ 格式化输出：字符串格式化%

符号	说明
%s	字符串
%c	字符
%d	(十进制) 整数
%u	无符号整数
%o	八进制数
%x	十六进制数
%X	十六进制数 (大写)
%e	用科学记数法表示浮点数
%E	同%e
%f	浮点数, 可指定小数点后的精度
%g	采用%f和%e较短者输出
%G	采用%f和%E较短者输出
%p	以十六进制数格式化变量地址
%%	字符" %"

- Python用一个元组将多个值传递给模板，每个值对应一个格式符。

```
In [26]: ▶ print('名字: %s, 年龄: %d' % ('张三', 20))
```

名字: 张三, 年龄: 20

- "%s speak plainer than %s."为格式化输出时的字符串模板。
- %s为一个格式符，表示一个字符串。元组('Facts', 'words')的两个元素'Facts'和'words'为分别替换第一个%s和第二个%s的真实值。
- 在模板和元组之间，有一个%号分隔，它代表了格式化操作。

```
In [111]: ▶ print("%s speak plainer than %s." % ('Facts', 'words'))
```

```
Facts speak plainer than words.
```

此外，格式化操作符还有如下辅助命令：

- **m . n** : **m**是数字的总宽度，**n**是小数位数；
- **-** : 用于左对齐；
- **+** : 在正数前面显示加号；
- **0** : 显示的数字前面填充'0'取代空格。

```
In [91]: ▶ print('%f' % 3.1415926)
          print('%20.2f' % 3.1415926)
```

3.141593

3.14

- 附加格式说明符：可以用如下的方式，对输出的格式字符进行进一步的控制：

'%[(name)][flags][width].[precision]type' %x

name可为空，对格式符进行命名，其为键名。

flags可以有+、-、' '或0。+在正数前面显示加号。-表示左对齐。' '为一个空格，表示在正数的左侧填充一个空格，从而与负数对齐。0表示使用0填充空位。

width表示显示的宽度。

precision表示小数点后精度。

type表示数据输出的格式类型。

● 字符串辅助操作符

符号	说明
*	字符串
-	左对齐
+	在正数前面显示加号
#	在八进制数前面显示0，在十六进制前面显示0x（或0X）
0	显示的数字签名填充0而不是默认空格
m.n	m是最小总宽度，n是小数点后位数

附加格式说明符举例：

```
In [112]: ▶ year=2021  
month=1  
day=15  
print(' %04d-%02d-%02d' %(year, month, day))  
  
2021-01-15
```

```
In [2]: ▶ print('这个商品的价格是 ￥%.2f' %7.8)  
  
这个商品的价格是 ￥7.80
```

附加格式说明符举例：

```
In [29]: ▶ value = 8.123  
print('%06.2f' % value)  
print('%02x' % 10)  
print('% .2e' % 1.2888)  
print("%6.3f%%" % 2.3)
```

008.12

0a

1.29e+00

2.300%

● 字符串的format方法

str.format()函数提供了比%方式更为强大的格式化方式，通过传入的参数进行格式化，并且使用大括号‘{}’作为特殊字符代替‘%’。

str.format()格式化输出使用花括号{}来包围替换字段，也就是待替换的字符串。而未被花括号包围的字符会原封不动地出现在输出结果中。

● 字符串的format方法

<模板字符串>.format(<逗号分隔的参数>)

<模板字符串>是由一系列的占位符组成，用来控制字符串中嵌入值的出现位置及格式，<逗号分隔的参数>按照序号顺序替换到<模板字符串>的占位符处。占位符如何被替换取决于每个格式说明符，格式说明符以“:”作为其前缀来表示。

占位符用大括号{}括起来，如果大括号中没有序号，则按照位置顺序替换。除了通过序号来指定填充的参数外，还可以通过关键字参数、下标、字典的key和对象的属性来填充。

● 字符串的format方法示例:

```
In [93]: ▶ print(' {}:计算机{}的CPU占用率为{}%.'.format('2019-01-30', 'Python', 10))
```

```
2019-01-30:计算机Python的CPU占用率为10%.
```

① 使用位置（序号）索引

以下两种写法是等价的：

```
In [30]: ▶ print("Hello, {} and {}".format("John", "Mary"))  
          print("Hello, {0} and {1}".format("John", "Mary"))
```

Hello, John and Mary!

Hello, John and Mary!

花括号内部可以写上待输出的目标字符串的索引，也可以省略。如果省略，则按format后面的括号里的待输出的目标字符串顺序依次替换。

```
In [31]: ▶ print('{1}{0}{1}'.format('言', '文'))
```

文言文

① 使用位置（序号）索引

使用“{序号: 格式说明符}”形式的格式说明符:

```
In [42]: ▶ print("{0:.2f}, {1}".format(3.1415926, 100))
```

```
3.14, 100
```

① 使用位置（序号）索引

使用“{序号/键: 格式说明符}”形式的格式说明符:

```
In [43]: ► print("{0}, pi={x}".format("圆周率", x=3.14))
```

```
圆周率, pi=3.14
```

② 使用关键字索引：

除了通过位置来指定待输出的目标字符串的索引，还可以通过关键字来指定待输出的目标字符串的索引。

```
In [32]: ► print("Hello, {boy} and {girl}!".format(boy="John", girl="Mary"))  
          print("{a} {b}".format(b="3", a="Python"))
```

```
Hello, John and Mary!  
Python3
```

③ 使用属性索引：

在使用`format()`来格式化字符串时，通常将目标字符串作为参数传递给`format`方法，此外还可以在格式化字符串中访问参数的某个属性，即使用属性索引。

```
In [33]: ► c = 3-5j  
print('复数{0}的实部为{0.real}，虚部为{0.imag}。').format(c))
```

复数(3-5j)的实部为3.0，虚部为-5.0。

④ 使用下标索引:

```
In [34]: ▶ coord = (3, 5, 7)
print('X: {0[0]}; Y: {0[1]}; Z: {0[2]}'.format(coord))

X: 3; Y: 5; Z: 7
```

- 字符串是一种**有序字符的集合**，用于表示**文本数据**。
- 字符串可以包含**零个、一个或者多个字符**。字符串使用**单引号'**、**双引号"**或**三引号（''' '''、'''''' '''）**作为字符串的界定符。
- 字符串可以使用**ASCII**字符、**Unicode**字符以及各种符号。

```
In [38]: ▶ var1="Hello World!"  
          print(var1)  
  
Hello World!
```

- **Python 3的字符串编码：可以通过代码查看Python 3的字符串默认编码。**

```
In [39]: ▶ import sys  
          sys.getdefaultencoding()
```

```
Out[39]: 'utf-8'
```

- 字符串**编码**和**解码**举例：编码和解码格式不一致，会导致程序错误。

```
In [77]: ► a='Python语言'.encode('utf8')  
          b='Python语言'.encode('gb2312')  
          c=b'Python\xe8\xaf\xad\xe8\xa8\x80'.decode('utf8')  
          print(a)  
          print(b)  
          print(c)
```

```
b'Python\xe8\xaf\xad\xe8\xa8\x80'  
b'Python\xd3\xef\xd1\xd4'  
Python语言
```

- 字符串**编码**和**解码**举例：编码和解码格式不一致，会导致程序错误。

```
In [81]: ► 'Python语言'.encode('gb2312').decode('utf8')
```

```
UnicodeDecodeError                                Traceback (most recent call last)
```

```
<ipython-input-81-271c0262ad5e> in <module>
```

```
----> 1 'Python语言'.encode('gb2312').decode('utf8')
```

```
      2 #编码和解码格式不一致，导致错误
```

```
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xd3 in position 6: invalid continuation byte
```

- Python 3.x完全支持中文字符，默认使用UTF8编码格式，无论是一个数字、英文字母，还是一个汉字，在统计字符串长度时都按一个字符对待和处理。
- 字符串属于不可变序列，意味着不可以直接修改字符串的值。

(1) 创建字符串

只要为变量分配一个用字符串界定符括起来的字符序列即可创建一个字符串。

```
In [38]: ▶ var1="Hello World!"  
print(var1)
```

```
Hello World!
```

- 字符串长度，中文字符和英文字符都算1个。

```
In [82]: ► len("Python语言")
```

```
Out[82]: 8
```

- 修改字符串变量的值会导致重新创建一个对象，并指向这个对象。

```
In [83]: ► s='mi'  
print(id(s))  
s = 'Python'  
print(id(s))
```

```
1373481710896
```

```
1373482759088
```

- 字符串类型在Python中是十分重要的类型，一般用引号中间添加字符的形式表达。
- 不同于其它语言，Python中双引号（" "）与单引号（' '）是不加区分的，都可以用来表示字符串，一般有如下三种表现方式：

1、使用单引号' '包含字符

`'Python'` `'abc'` `'中国'`

注意：单引号表示的字符串里面不能包含单引号，比如Let's go不能使用单引号包含。

2、使用双引号" " 包含字符

"PyCharm " "ABC " "北京"

注意：双引号表示的字符串里面不能包含双引号，并且只能有一行。

3、使用三引号（三对单引号''' '''或三对双引号""" """）包含字符

三引号'''或"""表示的字符串可以换行，支持排版较为复杂的字符串，包含在一对三引号之间而且不属于任何语句的内容被解释器认为是注释。

'''

hello,world!

'''

In [86]: ▶

```
'''  
我是多行注释，  
不要搞混了哦！  
'''  
  
print('''Python是一门编程语言  
Python能做数学运算  
要做好练习  
''')  
print('Python是一门编程语言\nPython能做数学运算\n要做好练习\n')
```

Python是一门编程语言
Python能做数学运算
要做好练习

Python是一门编程语言
Python能做数学运算
要做好练习

- 单引号、双引号、三单引号、三双引号可以互相嵌套，用来表示复杂字符串。例如：'''Jack said, "I'm very good" '''
- 空字符串表示为""或""。
- 前缀带r或者R的字符串表示Raw字符串（原始字符串）。

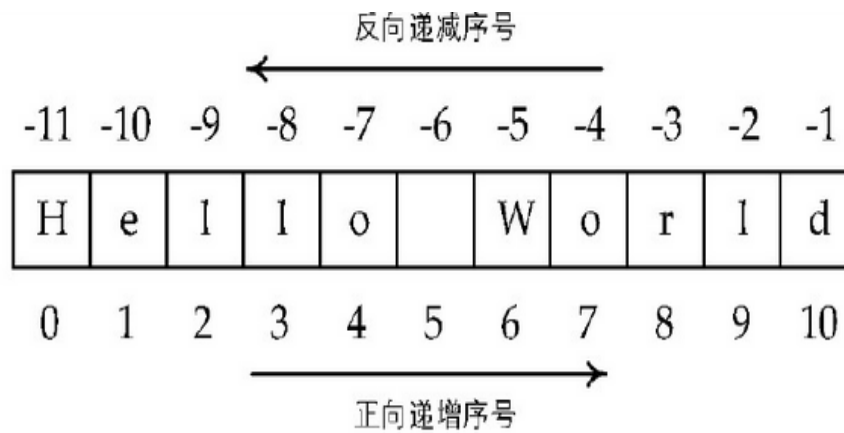
Raw字符串中的所有字符都被看做普通字符，不会处理其中的转义字符。但是，字符串的最后一个字符不能是“\”。原始字符串主要用于正则表达式、文件路径或者**URL**的场合。

```
In [75]: ▶ path = 'C:\Windows\twunk.exe'  
print(path)  
path1 = r'C:\Windows\twunk.exe'  
print(path1)
```

```
C:\Windows      wunk.exe  
C:\Windows\twunk.exe
```

字符串访问方式:

- 字符串包括正向递增序号和反向递减序号两种序号体系。
- 正向递增序号: $0 \sim L-1$, 以最左侧字符序号为0, 向右依次递增, 最右侧字符序号为 $L-1$ 。其中 L 为字符串的长度。
- 反向递减序号: $-L \sim -1$, 以最右侧字符序号为-1, 向左依次递减, 最左侧字符序号为 $-L$ 。



字符串访问方式:

- Python字符串提供**区间**访问方式，采用**[N: M]**格式，表示字符串中**从N到M（不包含M）**的子字符串，其中，**N和M**为字符串的**索引序号**，可以混合使用**正向递增序号**和**反向递减序号**。如果表示中**M**或者**N**索引缺失，则表示字符串把开始或结束索引值设为默认值。

字符串访问方式:

```
In [87]: ▶ str = "Python大数据基础与实战"  
print(str[0], str[5], str[-1])  
print(str[2:6])  
print(str[:6])  
print(str[6:])  
print(str[:])
```

P n 战

thon

Python

大数据基础与实战

Python大数据基础与实战

字符串的转义：

- 如果需要在字符中使用特殊字符时，Python用**反斜杠**。反斜杠字符是一个特殊字符，在字符串中表示“转义”。
- 当Python字符串中有一个反斜杠时表示一个转义序列的开始，称**反斜杠为转义符**。

转义字符

转义字符	含义	转义字符	含义
\\	反斜线	\a	响铃符
\'	单引号	\b	退格符
\"	双引号	\f	换页符
\n	换行	\r	回车符
\t	水平制表符	\v	垂直制表符
\ddd	八进制表示的ASCII码对应的字符	\xdd	十六进制表示的ASCII码对应的字符

字符串的转义：

- C语言把空字符“\0”作为字符串的结束标志，但是在Python中，空字符是作为一个普通字符进行处理。
- 以下是一个空字符、一个十六进制和一个八进制表示的ASCII字符。

```
In [88]: ▶ s = '\0\x61\101'
          print(s)
          print(len(s))

aA
3
```

字符串运算符

操作符	描述
+	字符串连接
*	重复输出字符串
[]	通过索引获取字符串中字符
[:]	截取字符串中的一部分
in	成员运算符--如果字符串中包含给定的字符串返回True
not in	成员运算符--如果字符串中不包含给定的字符串返回True
r/R	原始字符串--在字符串的第一个引号前加上字母r或R，字符串中的所有的字符直接按照字面的意思来使用，不再转义特殊或不能打印的字符。
%	格式化字符串

字符串运算符:

- Python提供了5个字符串的基本操作符，如表所示。

操作符	描述
<code>x + y</code>	连接两个字符串x与y
<code>x * n</code> 或 <code>n * x</code>	复制n次字符串x
<code>x in s</code>	如果x是s的子串，返回True，否则返回False
<code>str[i]</code>	索引，返回第i个字符
<code>str[N: M]</code>	切片，返回索引第N到M的子串，其中不包含M

字符串连接操作

(1) 基本连接操作+

$s1+s2+\cdots+sn$

其中, $s1, s2, \cdots sn$, 均是一个字符串, 表达式的值也是一个字符串。

(2) 重复连接操作 $s*n$ 或 $n*s$

```
In [89]: ► s1 = "Hello" + " " + "Python"
          print(s1)
          s2 = "重要的事情说三遍!"*3
          print(s2)
          s3 = 'Python' in s1
          print(s3)
```

Hello Python

重要的事情说三遍! 重要的事情说三遍! 重要的事情说三遍!

True

字符串的索引

Python中的字符串有两种索引方式，从左往右以0开始，从右往左以-1开始。字符串变量名后接用中括号括起来的编号即可实现字符串的索引。

s[0]	s[1]	s[2]	s[3]	s[4]
H	e	l	l	o
s[-5]	s[-4]	s[-3]	s[-2]	s[-1]

In [47]:



```
s="Hello"  
print(s[0], s[-4])
```

H e

● 字符串的分片

字符串的分片(或切片)就是从给定的字符串中分离出部分字符, 这时可以使用以下形式的字符串索引编号:

`str[i:j:k]`

其中, i是索引起始位置, j是索引结束位置但**不包括位置上的字符**, 索引编号每次增加的步长为k。

```
In [50]: ▶ str1="Python"
          print (str1[0])           # 通过索引输出字符串第一个字符
          print (str1[2:5])         # 输出从第三个开始到第五个的字符
          print (str1[0:-1])        # 输出第一个到倒数第二个的所有字符
          print(str1[1:5:2])
```

```
P
tho
Pytho
yh
```

● 字符串的逻辑操作

字符串的逻辑操作是指字符串参与逻辑比较，其操作的结果是一个逻辑量，通常用于表达字符处理的条件。

(1) 关系操作（比较）

(2) 成员关系操作

字符串的成员关系操作包括in和not in操作，一般格式为：字符串1 [not] in字符串2

该操作用于判断字符串1是否属于字符串2，其返回值为True或False。

● 字符串的逻辑操作

```
In [51]: ▶ print("SHANGHAI"<"SHANKONG")  
          print('a' in 'abc')
```

True

True

● r/R原始字符串

前缀带r或者R的字符串表示Raw字符串（原始字符串）。

Raw字符串中的所有字符都被看做普通字符，不会处理其中的转义字符。但是，字符串的最后一个字符不能是“\”。原始字符串主要用于正则表达式、文件路径或者URL的场合。

● r/R原始字符串例子:

```
In [75]: ▶ path = 'C:\Windows\twunk.exe'
          print(path)
          path1 = r'C:\Windows\twunk.exe'
          print(path1)
```

```
C:\Windows      twunk.exe
C:\Windows\twunk.exe
```

- 字符串作为常用的一种数据类型，Python提供了很多内建方法，如字符串的查找、索引、取长度、统计、替换和分割等。
- 方法的引用形式：对象名.方法名(参数)

字符串对象常用的方法

方法/函数	作用
str.capitalize()	返回字符串的副本，其首字符大写，其余字符小写
str.count(sub [,start [,end]])	返回[start, end]范围内sub的非重叠出现次数，start和end可选
str.endswith(sub[,start[,end]])	返回布尔值，表示字符串是否以指定的sub结束，同类方法str.startswith()
str.find(sub [,start [,end]])	返回字符串中首次出现子串sub的索引位置，start和end可选，若未找到sub，返回-1
str.split(sep =None)	使用sep作为分隔符拆分字符串，返回字符串中单词的列表，分隔空字符串
str.strip([chars])	删除字符串前端和尾部chars指定的字符集，如果省略或None，则删除空白字符

1. 字符串查找

- **find()**和**rfind()**函数分别用来查找一个字符串在另一个字符串指定范围（默认是整个字符串）中首次和最后一次出现的位置，如果不存在则返回-1；
- **index()**和**rindex()**方法用来返回一个字符串在另一个字符串指定范围中首次和最后一次出现的位置，如果不存在则抛出异常；
- **count()**方法用来返回一个字符串在当前字符串中出现的次数。

1. 字符串查找示例:

```
In [94]: ► mystr = "Python is an excellent language"  
          index = mystr.find("programming")  
          index1 = mystr.find("excellent", 0, 30)  
          print(index)  
          print(index1)
```

```
-1  
13
```

1. 字符串查找示例:

```
In [97]: ▶ str = "i love python,i am learning python"  
          #star 和end 为默认参数  
          print(str.count("i"))  
          # star值为2, end值为默认参数  
          print(str.count("i",2))
```

3

2

2. 字符串分割

- `split()`、`rsplit()`方法分别用来以指定字符为分隔符，把当前字符串从左往右、从右往左分隔成多个字符串，并返回包含分隔结果的**列表**。
- 对于`split()`和`rsplit()`方法，如果不指定分隔符，则字符串中的任何空白符号（空格、换行符、制表符等）都将被认为是分隔符，把连续多个空白字符看作一个分隔符。
- `partition()`和`rpartition()`用来以指定字符串为分隔符将原字符串分隔为3部分：分隔符前的字符串、分隔符字符串、分隔符后的字符串。如果指定的分隔符不在原字符串中，则返回原字符串和两个空字符串。

2. 字符串分割示例:

```
In [96]: ▶ str = 'WinXP||Win7||Win8||Win10'  
print (str.split('||'))  
#指定最大分隔次数为2  
print (str.split('||', 2))→  
print(str.partition("||"))
```

```
['WinXP', 'Win7', 'Win8', 'Win10']
```

```
['WinXP', 'Win7', 'Win8||Win10']
```

```
('WinXP', '||', 'Win7||Win8||Win10')
```

3. 字符串的连接

- `join()`方法用来将列表中的多个字符串进行连接，并且在相邻两个字符串之间插入指定字符。示例如下：

```
In [98]: ► test = ["I", "love", "Python"]  
s = " ".join(test) → #指定插入字符为空格  
print(s)
```

I love Python

- 注意：使用“+”运算符也可以连接字符串，但是效率较低，应优先使用`join()`方法。

4. 字符串的大小写转换

- **lower()**返回小写字符串，**upper()**返回大写字符串，**capitalize()**将字符串首字符大写、**title()**将字符串中每个单词首字符大写、**swapcase()**完成大小写互换。

4. 字符串的大小写转换示例:

```
In [99]: ► s = "i am a teacher"  
print(s.lower())  
print(s.upper())  
print(s.capitalize())  
print(s.title())  
print(s.swapcase())
```

```
i am a teacher  
I AM A TEACHER  
I am a teacher  
I Am A Teacher  
I AM A TEACHER
```

5. 字符串替换

- `replace()` 方法把字符串中的 `old`（旧字符串）替换成 `new`(新字符串)，如果指定第三个参数`max`，则替换不超过 `max` 次。

```
In [100]: ► str = "this is string example....wow!!! this is really string"
print (str.replace("is", "was"))
print (str.replace("is", "was", 3))
```

```
thwas was string example....wow!!! thwas was really string
thwas was string example....wow!!! thwas is really string
```

6. 字符串删除

- **strip()**删除字符串两端指定的字符，**rstrip()**删除字符串右端指定字符，**lstrip()**删除字符串左端指定字符。

```
In [101]: ► s = " he is a student\t\t"
           #删除s左右两端的空白字符
           print(s.strip())
           print(s.rstrip("\t"))
           'aabbccddeeefg'.strip('gaef')
```

```
he is a student
he is a student
```

```
Out[101]: 'bbccdd'
```

7. 字符串测试

- `isalnum()`、`isalpha()`、`isdigit()`、`isdecimal()`、`isnumeric()`、`isspace()`、`isupper()`、`islower()`，分别测试字符串是否为数字或字母、是否为字母、是否为数字字符、是否为空白字符、是否为大写字母以及是否为小写字母。

```
In [103]: ► print('1234abcd'.isalnum())  
           #全部为英文字母时返回True  
           print('1234abcd'.isalpha())  
           #全部为数字时返回True  
           print('1234abcd'.isdigit())
```

```
True  
False  
False
```

8. eval()

- 内置函数eval()尝试把任意字符串转化为Python表达式并求值。

```
In [104]: ► eval(" 10 * 2 / 5")
```

```
Out[104]: 4.0
```

9. startswith()、endswith()

- 这两个方法用来判断字符串是否以指定字符串开始或者结束。

```
In [105]: ▶ print("test.py".endswith((".py", ".cpp", ".java")))
           print("test.py".startswith("test", 0))

True
True
```

10. center()、ljust()、rjust()

- 返回指定宽度的新字符串，原字符串居中、左对齐或右对齐出现在新字符串中，如果指定的宽度大于字符串长度，则使用指定的字符（默认为空格）填充。

```
In [106]: ▶ print("let's begin".center(20, "+"))  
           print("let's begin".ljust(20, "-"))
```

```
++++let's begin++++  
let's begin-----
```

- 字符串对象常用的方法

- (1) 字母大小写转换

- s.upper():** 全部转换为大写字母。

- s.lower():** 全部转换为小写字母。

- s.swapcase():** 字母大小写互换。

- s.capitalize():** 句子首字母大写，其余小写。

- s.title():** 单词首字母大写。

- 其中，s是一个字符串对象。

● 字符串对象常用的方法

(2) 字符串搜索

s.find(substr,[start,[end]]): 返回s中出现substr的第1个字符的编号，如果s中没有substr则返回-1。start和end作用就相当于在s[start:end]中搜索。

s.index(substr,[start,[end]]): 与find()相同，只是在s中没有substr时，会返回一个运行错误。

s.rfind(substr,[start,[end]]): 返回s中最后出现的substr的第1个字符的编号，如果s中没有substr则返回-1，也就是说从右边算起的第1次出现的substr的首字符编号。

● 字符串对象常用的方法

(2) 字符串搜索

s.rindex(substr,[start,[end]]): 与rfind()相同, 只是在s中没有substr时, 会返回一个运行时错误。

s.count(substr,[start,[end]]): 计算substr在s中出现的次数。

s.startswith(prefix,[start,[end]]): 是否以prefix开头, 若是返回True, 否则返回False。

s.endswith(suffix,[start,[end]]): 以suffix结尾, 若是返回True, 否则返回False。

● 字符串对象常用的方法

(3) 字符串替换

s.replace(oldstr,newstr,[count]): 把s中的oldster替换为newstr, count为替换次数。这是替换的通用形式, 还有一些函数进行特殊字符的替换。

s.strip([chars]): 把s中前后chars中有的字符全部去掉, 可以理解为把s前后chars替换为None。默认去掉前后空格。

s.lstrip([chars]): 把s左边chars中有的字符全部去掉。默认去掉左边空格。

s.rstrip([chars]): 把s右边chars中有的字符全部去掉。默认去掉右边空格。

s.expandtabs([tabsize]): 把s中的tab字符替换为空格, 每个tab替换为tabsize个空格, 默认是8个。

● 字符串对象常用的方法

(4)字符串的拆分与组合

s.split([sep,[maxsplit]]): 以sep为分隔符，把字符串s拆分成一个列表。默认的分隔符为空格。**maxsplit**表示拆分的次数，默认取-1，表示无限制拆分。

s.rsplit([sep,[maxsplit]]): 从右侧把字符串s拆分成一个列表。

s.splitlines([keepends]): 把s按行拆分为一个列表。**keepends**是一个逻辑值，如果为True，则每行拆分后会保留行分隔符。

● 字符串对象常用的方法

(4)字符串的拆分与组合

s.partition(sub): 从sub出现的第1个位置起, 把字符串s拆分成一个3元素的元组(sub左边字符, sub, sub右边字符)。如果s中不包含sub, 则返回(s, "", "")。

s.rpartition(sub): 从右侧开始, 把字符串s拆分成一个3元素的元组(sub左边字符, sub, sub右边字符)。如果s中不包含sub, 则返回("", "", s)。

s.join(seq): 把seq代表的序列组合成字符串, 用s将序列各元素连接起来。字符串中的字符是不能修改的, 如果要修改, 通常的方法是, 用list()函数把字符串s变为以单个字符为成员的列表(使用语句s=list(s)), 再使用给列表成员赋值的方式改变值(如s[3]='a'), 最后再使用语句“s=""**.join(s)**”还原成字符串。

● 字符串对象常用的方法

(5) 字符串类型测试

字符串类型测试函数返回的都是逻辑值。

s.isalnum(): 是否全是字母和数字，并至少有一个字符。

s.isalpha(): 是否全是字母，并至少有一个字符。

s.isdigit(): 是否全是数字，并至少有一个字符。

s.isspace(): 是否全是空格，并至少有一个空格。

s.islower(): s中的字母是否全是小写。

s.isupper(): s中的字母是否全是大写。

s.istitle(): s是否为首字母大写。